

Project Documentation
BlackBoxESP

Filip Radojević

February 20, 2026

Contents

1	Introduction	2
2	System Architecture	3
2.1	Communication Configuration	3
2.2	System Initialization	3
2.2.1	Black Box State Machine	4
2.3	Task Architecture	5
2.3.1	Work Task	5
2.3.2	Log Task	5
2.4	System Behavior Overview	6
3	Project Structure	7
3.1	Development Environment	7
4	System Diagram	8

Chapter 1

Introduction

The project was initially developed on LPC1768 boards used in the laboratory, where several system modules were implemented, such as `GW_GND`, `GW_SKY`, `INS`, `INS_COST`, `POWER_MANAGEMENT`, `ACTUATOR_MASTER`, and the current `BlackBox`.

However, due to higher performance requirements, the project is now being migrated to the ESP32-S3(R8) ETH Kit, which provides significantly larger memory: both SRAM for network buffers (allowing larger `pcache` and `rcache` sizes in LFS when using larger flash `page_size`), and PSRAM which is intended to serve as a ring buffer for data logging.

Chapter 2

System Architecture

2.1 Communication Configuration

The system supports both **Cyphal** and **MAVLink** communication protocols. The selection is performed at compile time using a preprocessor macro defined in `types.h`. This design allows flexible switching between communication stacks without modifying the core application logic.

2.2 System Initialization

The `main.c` file is responsible solely for low-level system initialization. It configures all necessary hardware peripherals and operating system components, including:

- Real-Time Clock (RTC)
- PSRAM-based Ring Buffer for inter-task data exchange
- FreeRTOS tasks and synchronization objects
- Ethernet or CAN peripheral (depending on configuration)
- Storage interface (SD Card or internal Flash)
- File system initialization (LittleFS)

The Real-Time Clock is used to provide precise time for name of our file.

The current file system implementation is based on **LittleFS (LFS)**, a lightweight and fail-safe filesystem designed for embedded systems. More information about LittleFS can be found at:

<https://github.com/littlefs-project/littlefs>

For data logging, the system uses the **ULog** file format, which provides a structured and efficient binary logging format. Further documentation is available at:

https://docs.px4.io/main/en/dev_log/ulog_file_format

2.2.1 Black Box State Machine

An internal state machine is implemented specifically for the **BLACK_BOX** node. This state machine determines system behavior and logging policy.

The defined states are:

- **Initialization** – Short startup phase after system boot.
- **Idle** – Waiting for commands from the Ground Application.
- **Operational** – Active logging mode; all bus traffic is recorded.
- **Maintenance** – Logging disabled; system prepared for data extraction.
- **Software Update** – Reserved for firmware update procedures.

After completing hardware and system initialization in **main.c**, the system transitions from **Initialization** to **Idle**.

In the **Idle** state, the device waits for commands from the Ground Application, which may request:

- Activation of logging mode (Operational)
- Extraction of stored log files via FTP

In **Operational** mode, a new log file is created and all relevant bus traffic is recorded.

The **Maintenance** state is used when log files need to be extracted and transmitted to the Ground Station. During this phase, devices may enter low-activity mode to ensure safe data handling.

2.3 Task Architecture

The current implementation consists of two FreeRTOS tasks:

- **work**
- **log**

Both tasks operate at the same priority level.

2.3.1 Work Task

The **work** task is responsible for communication processing. Incoming frames from Ethernet or CAN are processed using:

- `udp_process()` (for MAVLink over Ethernet)
- `cyphal_process()` (for Cyphal over CAN)

When a received frame matches one of the registered subscriptions, the corresponding payload is forwarded to a dedicated handler.

The handler performs the following operations:

- Extracts the payload
- Stores the data into the PSRAM Ring Buffer
- Appends metadata required for message identification

2.3.2 Log Task

The **log** task continuously monitors the PSRAM Ring Buffer. Its responsibilities include:

- Collecting buffered data
- Adding ULog message headers and structure markers
- Writing formatted data to persistent storage using LittleFS

The data is physically stored on either:

- SD Card
- Internal Flash memory

2.4 System Behavior Overview

In summary:

1. `main.c` initializes hardware, storage, and RTOS objects.
2. The `work` task processes incoming communication frames.
3. Valid payloads are buffered in PSRAM with identification metadata.
4. The `log` task formats the data into ULog structure.
5. Data is written to non-volatile memory via LittleFS.

This architecture ensures deterministic communication handling, safe buffering between tasks, structured logging, and reliable persistent storage in an embedded real-time environment.

This ensures that the log task immediately handles data logging without wasting CPU cycles. The log task performs `lfs_write` operations to store data into flash. With this approach, the system has achieved a maximum reliable logging speed of **1.2 Mbit/s**.

However, due to the slow synchronization mechanism of `lfs_sync`, which is necessary to ensure power-loss safety, a larger ring buffer is required to avoid data corruption or loss. Therefore, PSRAM is used as a large intermediate buffer, ensuring that writing speed can exceed reading speed without compromising system integrity.

Chapter 3

Project Structure

The project is organized into the following modules:

- **app/** – main application (e.g., `task_work.c`, `task_log.c`, `main.c`)
- **drivers/** – hardware drivers (Ethernet, flash, SD card, FTP, etc.)
- **middleware/** – protocols and layers (UDP, MAVLink, Cyphal ULog, LFS, etc.)
- **libs/** – Libcanard if cyphal is used

3.1 Development Environment

The project is developed in **Visual Studio Code**, where the build system is integrated with the ESP-IDF (Espressif IoT Development Framework) toolchain and `ninja` build system.

- **F7** – Compile (Build process)
- **F3** – Clean Rebuild
- **F5** – Flash the project to the microcontroller

Chapter 4

System Diagram

The following diagram illustrates the workflow of the system, showing how the **Work Task**, **Log Task**, PSRAM ring buffer, and Flash interact with each other through semaphores and data pipelines.

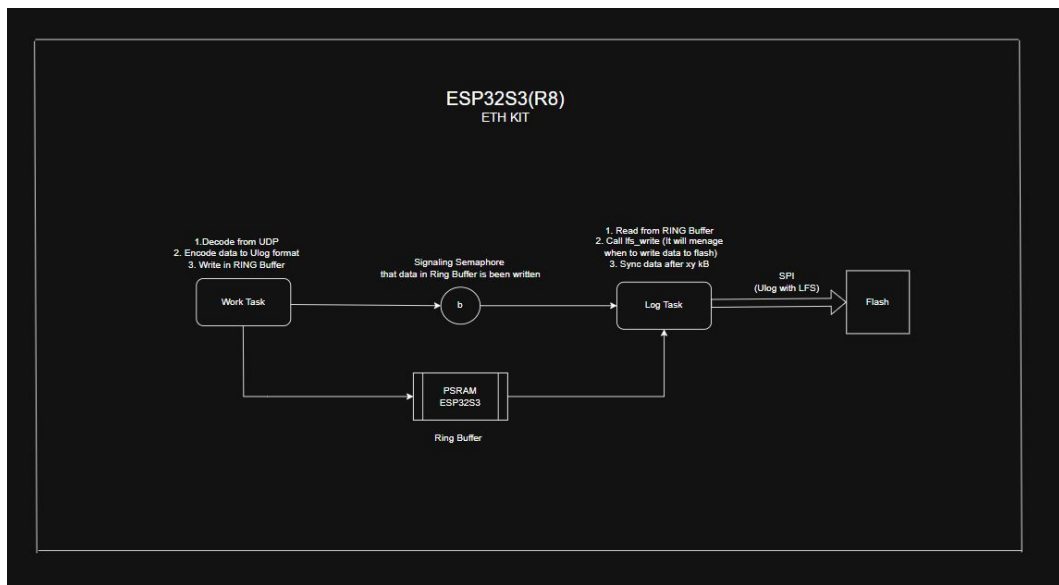


Figure 4.1: System architecture of ESP32-S3(R8) ETH Kit implementation.